MODULE *DataLakehouseGateway*

EXTENDS *Naturals*, *Sequences*, *FiniteSets*, *TLC*

CONSTANTS
Tenants, *Clients*, *Requests*, *Queries*, *Workers*, *Blobs*, *ProxyNodes*,
MaxSnapshot, *MaxAttempts*, *MaxCrashes*, *MaxTimeouts*, *ScaleOutThreshold*,
RequestedTenantOf,
AuthenticatedTenantOf,
QueryOf, *ClientOf*,
ProtocolOf,
SecurityContextOf,
FlightPopulatesCache,
MutantSkipTenantCheck

ASSUME
$\land$ DOMAIN *RequestedTenantOf* $=$ *Requests*
$\land$ DOMAIN *AuthenticatedTenantOf* $=$ *Requests*
$\land$ DOMAIN *QueryOf* $=$ *Requests*
$\land$ DOMAIN *ClientOf* $=$ *Requests*
$\land$ DOMAIN *ProtocolOf* $=$ *Requests*
$\land$ DOMAIN *SecurityContextOf* $=$ *Requests*
$\land$ *ProxyNodes* $\neq \{\}$
$\land$ *ScaleOutThreshold* $\in$ *Nat*
$\land$ *MaxSnapshot* ≥ 1 $\land$ *MaxAttempts* ≥ 1
$\land$ *MaxCrashes* $\in$ *Nat* $\land$ *MaxTimeouts* $\in$ *Nat*
$\land$ *FlightPopulatesCache* $\in$ BOOLEAN
$\land$ *MutantSkipTenantCheck* $\in$ BOOLEAN
$\land$ *Requests* $\cap$ *Workers* $= \{\}$

NONE $\triangleq$ "NONE"

SecCtxs $\triangleq$ $\{$*SecurityContextOf*$[r] : r \in$ *Requests*$\}$
NoEntry $\triangleq$ $[$*valid* $\mapsto$ FALSE, *blob_id* $\mapsto$ *NONE*$]$
FreeBlob $\triangleq$ [
allocated $\mapsto$ FALSE,
staged $\mapsto$ FALSE,
req $\mapsto$ *NONE*,
epoch $\mapsto 0$,
tenant $\mapsto$ *NONE*,
query $\mapsto$ *NONE*,

$version \mapsto 0,$
$security_context \mapsto NONE$
]

--algorithm *DataLakehouseGateway*

variables
$hash_ring \in [Tenants \times Queries \to ProxyNodes],$
$catalog_snapshot = [t \in Tenants \mapsto 1],$
$cache = [p \in ProxyNodes \mapsto$
$[t \in Tenants \mapsto$
$[q \in Queries \mapsto$
$[v \in 1 \,..\, MaxSnapshot \mapsto$
$[s \in SecCtxs \mapsto NoEntry]]]]],$
$task_queue = \langle\rangle,$
$jobs = [r \in Requests \mapsto$
$[state \mapsto$ "IDLE",
$owner \mapsto NONE,$
$lease_epoch \mapsto 0,$
$attempts \mapsto 0,$
$requested_tenant \mapsto RequestedTenantOf[r],$
$tenant \mapsto NONE,$
$query \mapsto QueryOf[r],$
$security_context \mapsto NONE,$
$version \mapsto 0,$
$blob_ptr \mapsto NONE,$
$protocol \mapsto ProtocolOf[r],$
$rls_applied \mapsto$ FALSE,
$executed_predicate_tenant \mapsto NONE]],$
$blob_storage = [b \in Blobs \mapsto FreeBlob],$
$available_blobs = Blobs,$
$worker_status = [w \in Workers \mapsto$ "OFFLINE"],
$client_received = [r \in Requests \mapsto$
$[delivered \mapsto$ FALSE,
$released \mapsto$ FALSE,
$blob_id \mapsto NONE,$
$tenant \mapsto NONE,$
$query \mapsto NONE,$
$version \mapsto 0,$
$security_context \mapsto NONE]],$
$commit_count = [r \in Requests \mapsto 0],$
$lease_expired = [r \in Requests \mapsto$ FALSE],
$crashes = 0,\ timeouts = 0$ **;**

define

$Pending \triangleq \{\text{“QUEUED”}, \text{“CLAIMED”}, \text{“RUNNING”}, \text{“STAGED”}\}$
$Active \triangleq \{\text{“CLAIMED”}, \text{“RUNNING”}, \text{“STAGED”}\}$
$Terminal \triangleq \{\text{“ACCEPTED”}, \text{“FAILED”}, \text{“UNAUTHORIZED”}, \text{“CACHED”}\}$
$AllStates \triangleq \{\text{“IDLE”}\} \cup Pending \cup Terminal$

$EffTenant(r) \triangleq$
IF $MutantSkipTenantCheck$
THEN $RequestedTenantOf[r]$
ELSE $AuthenticatedTenantOf[r]$

$HoldsLease(r, e, w, st) \triangleq$
$\land\ jobs[r].lease_epoch = e$
$\land\ jobs[r].owner = w$
$\land\ jobs[r].state = st$

$QCount(r) \triangleq Cardinality(\{i \in 1 \,..\, Len(task_queue) : task_queue[i] = r\})$

$BlobMatches(b, t, q, v, s) \triangleq$
$\land\ blob_storage[b].tenant = t$
$\land\ blob_storage[b].query = q$
$\land\ blob_storage[b].version = v$
$\land\ blob_storage[b].security_context = s$

$BlobCached(b) \triangleq$
$\exists\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1 \,..\, MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \land cache[p][t][q][v][s].blob_id = b$

$BlobJobHeld(b) \triangleq$
$\exists\, r \in Requests : jobs[r].blob_ptr = b \land \neg client_received[r].released$

$BlobClientHeld(b) \triangleq$
$\exists\, r \in Requests : client_received[r].delivered$
$\land\ \neg client_received[r].released$
$\land\ client_received[r].blob_id = b$

$StagedOrphan(b) \triangleq jobs[blob_storage[b].req].lease_epoch \neq blob_storage[b].epoch$

$GCable \triangleq \{b \in Blobs :$
$\land\ blob_storage[b].allocated$
$\land$ IF $blob_storage[b].staged$
THEN $StagedOrphan(b)$
ELSE $\neg BlobCached(b) \land \neg BlobJobHeld(b) \land \neg BlobClientHeld(b)\}$

$ExpirableJobs \triangleq \{r \in Requests :$

$\land\ jobs[r].state \in Active$
$\land\ (worker_status[jobs[r].owner] \neq \text{“ACTIVE”} \lor lease_expired[r])\}$

$CachedKeys \triangleq \{k \in ProxyNodes \times Tenants \times Queries \times (1\ ..\ MaxSnapshot) \times SecCtxs :$
$cache[k[1]][k[2]][k[3]][k[4]][k[5]].valid\}$

$Job_Unauthorized(j) \triangleq$
$[j\ \text{EXCEPT}\ !.state = \text{“UNAUTHORIZED”}]$
$Job_Cached(j) \triangleq$
$[j\ \text{EXCEPT}\ !.state = \text{“CACHED”}]$

$Job_Init(j,\ t,\ sc,\ v) \triangleq$
$[j\ \text{EXCEPT}\ !.tenant = t,\ !.security_context = sc,\ !.version = v]$

$Job_Flight_Accepted(j,\ b) \triangleq$
$[j\ \text{EXCEPT}\ !.state = \text{“ACCEPTED”},\ !.lease_epoch = 1,\ !.attempts = 1,$
$!.blob_ptr = b,\ !.rls_applied = \text{TRUE},\ !.executed_predicate_tenant = j.tenant]$

$Job_Enqueue(j) \triangleq$
$[j\ \text{EXCEPT}\ !.state = \text{“QUEUED”},\ !.attempts = j.attempts + 1]$

$Job_Claim(j,\ w) \triangleq$
$[j\ \text{EXCEPT}\ !.owner = w,\ !.state = \text{“CLAIMED”},\ !.lease_epoch = j.lease_epoch + 1]$

$Job_Run(j) \triangleq$
$[j\ \text{EXCEPT}\ !.state = \text{“RUNNING”},\ !.rls_applied = \text{TRUE},$
$!.executed_predicate_tenant = j.tenant]$

$Job_Stage(j,\ b) \triangleq$
$[j\ \text{EXCEPT}\ !.blob_ptr = b,\ !.state = \text{“STAGED”}]$

$Job_Accept(j) \triangleq$
$[j\ \text{EXCEPT}\ !.state = \text{“ACCEPTED”},\ !.owner = NONE]$

$Job_Retry(j) \triangleq$
$[j\ \text{EXCEPT}\ !.owner = NONE,\ !.state = \text{“QUEUED”},\ !.attempts = j.attempts + 1,$
$!.lease_epoch = j.lease_epoch + 1,\ !.blob_ptr = NONE,$
$!.rls_applied = \text{FALSE},\ !.executed_predicate_tenant = NONE]$

$Job_Fail(j) \triangleq$
$[j\ \text{EXCEPT}\ !.owner = NONE,\ !.state = \text{“FAILED”},\ !.lease_epoch = j.lease_epoch + 1,$
$!.blob_ptr = NONE,\ !.rls_applied = \text{FALSE},\ !.executed_predicate_tenant = NONE]$

$Client_Deliver(cr,\ b,\ t,\ q,\ v,\ sc)\ \triangleq$
$[cr\ \text{EXCEPT}\ !.delivered = \text{TRUE},\ !.blob_id = b,\ !.tenant = t,$
$!.query = q,\ !.version = v,\ !.security_context = sc]$

$Client_Release(cr)\ \triangleq$
$[cr\ \text{EXCEPT}\ !.released = \text{TRUE}]$

$Blob_Flight(t,\ q,\ v,\ sc)\ \triangleq$
$[allocated \mapsto \text{TRUE},\ staged \mapsto \text{FALSE},\ req \mapsto NONE,\ epoch \mapsto 0,$
$tenant \mapsto t,\ query \mapsto q,\ version \mapsto v,\ security_context \mapsto sc]$

$Blob_Stage(req,\ ep,\ t,\ q,\ v,\ sc)\ \triangleq$
$[allocated \mapsto \text{TRUE},\ staged \mapsto \text{TRUE},\ req \mapsto req,\ epoch \mapsto ep,$
$tenant \mapsto t,\ query \mapsto q,\ version \mapsto v,\ security_context \mapsto sc]$

$Blob_Commit(b_val)\ \triangleq$
$[b_val\ \text{EXCEPT}\ !.staged = \text{FALSE}]$

$ValidCache(b)\ \triangleq$
$[valid \mapsto \text{TRUE},\ blob_id \mapsto b]$

$InvalidateCache(old_cache,\ t,\ new_v)\ \triangleq$
$[p \in ProxyNodes \mapsto$
$[old_cache[p]\ \text{EXCEPT}\ ![t] =$
$[q \in Queries \mapsto$
$[v \in 1\ldots MaxSnapshot \mapsto$
$[s \in SecCtxs \mapsto$
$\text{IF}\ v < new_v$
$\text{THEN}\ NoEntry$
$\text{ELSE}\ \ old_cache[p][t][q][v][s]]]]]]$

$TypeOK\ \triangleq$
$\land\ catalog_snapshot \in [Tenants \to 1\ldots MaxSnapshot]$
$\land\ worker_status \in [Workers \to \{\text{“OFFLINE”},\ \text{“ACTIVE”},\ \text{“CRASHED”}\}]$
$\land\ hash_ring \in [Tenants \times Queries \to ProxyNodes]$
$\land \forall\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1\ldots MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \in \text{BOOLEAN}$
$\land \forall\, r \in Requests :$
$\land\ jobs[r].state \in AllStates$
$\land\ jobs[r].attempts \in 0\ldots MaxAttempts$

$\land\ jobs[r].lease_epoch \in Nat$
$\land\ jobs[r].blob_ptr \in Blobs \cup \{NONE\}$
$\land\ commit_count[r] \in 0\,..\,1$
$\land\ client_received[r].delivered \Rightarrow client_received[r].blob_id \neq NONE$

$TenantBinding \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state \notin \{\text{“IDLE”},\ \text{“UNAUTHORIZED”},\ \text{“CACHED”}\} \Rightarrow$
$\land\ jobs[r].tenant = AuthenticatedTenantOf[r]$
$\land\ jobs[r].security_context = SecurityContextOf[r]$

$RLSEnforcement \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state \in \{\text{“RUNNING”},\ \text{“STAGED”},\ \text{“ACCEPTED”}\} \Rightarrow$
$\land\ jobs[r].rls_applied$
$\land\ jobs[r].executed_predicate_tenant = AuthenticatedTenantOf[r]$

$UnauthorizedIsInert \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state = \text{“UNAUTHORIZED”} \Rightarrow$
$\land\ jobs[r].blob_ptr = NONE$
$\land\ jobs[r].owner = NONE$
$\land\ jobs[r].attempts = 0$
$\land\ commit_count[r] = 0$
$\land\ \neg client_received[r].delivered$
$\land\ QCount(r) = 0$

$MismatchRejected \triangleq$
$\forall\, r \in Requests :$
$RequestedTenantOf[r] \neq AuthenticatedTenantOf[r] \Rightarrow$
$\land\ jobs[r].state \in \{\text{“IDLE”},\ \text{“UNAUTHORIZED”}\}$
$\land\ \neg client_received[r].delivered$

$TenantIsolation \triangleq$
$\forall\, r \in Requests :$
$client_received[r].delivered \Rightarrow$
$\land\ client_received[r].tenant = AuthenticatedTenantOf[r]$
$\land\ client_received[r].security_context = SecurityContextOf[r]$
$\land\ client_received[r].query = QueryOf[r]$

$DeliveredResultIntegrity \triangleq$
$\forall\, r \in Requests :$
$(client_received[r].delivered \land \neg client_received[r].released) \Rightarrow$
$\land\ blob_storage[client_received[r].blob_id].allocated$
$\land\ \neg blob_storage[client_received[r].blob_id].staged$

$\land\ BlobMatches(client_received[r].blob_id,$
$AuthenticatedTenantOf[r],\ QueryOf[r],$
$client_received[r].version,\ SecurityContextOf[r])$

$ClaimCheckIntegrity\ \triangleq$
$\forall\, r \in Requests :$
$(jobs[r].state = \text{“ACCEPTED”} \land \neg client_received[r].released) \Rightarrow$
$\land\ blob_storage[jobs[r].blob_ptr].allocated$
$\land \neg blob_storage[jobs[r].blob_ptr].staged$
$\land\ BlobMatches(jobs[r].blob_ptr,\ jobs[r].tenant,\ jobs[r].query,$
$jobs[r].version,\ jobs[r].security_context)$

$BlobPtrIffResultState\ \triangleq$
$\forall\, r \in Requests :$
$(jobs[r].blob_ptr \neq NONE) \equiv$
$(jobs[r].state \in \{\text{“STAGED”},\ \text{“ACCEPTED”}\})$

$NoSharedBlobPointer\ \triangleq$
$\forall\, r1,\ r2 \in Requests :$
$(r1 \neq r2 \land jobs[r1].blob_ptr \neq NONE \land jobs[r2].blob_ptr \neq NONE$
$\land \neg client_received[r1].released \land \neg client_received[r2].released) \Rightarrow$
$jobs[r1].blob_ptr \neq jobs[r2].blob_ptr$

$BlobPoolConsistency\ \triangleq$
$available_blobs = \{b \in Blobs : \neg blob_storage[b].allocated\}$

$StagedBlobWellFormed\ \triangleq$
$\forall\, b \in Blobs :$
$blob_storage[b].staged \Rightarrow$
$\land\ blob_storage[b].allocated$
$\land\ blob_storage[b].req \in Requests$
$\land\ jobs[blob_storage[b].req].lease_epoch \geq blob_storage[b].epoch$

$CacheIntegrity\ \triangleq$
$\forall\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \Rightarrow$
$\land\ blob_storage[cache[p][t][q][v][s].blob_id].allocated$
$\land \neg blob_storage[cache[p][t][q][v][s].blob_id].staged$
$\land\ BlobMatches(cache[p][t][q][v][s].blob_id,\ t,\ q,\ v,\ s)$

$CacheFreshness\ \triangleq$
$\forall\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \Rightarrow v = catalog_snapshot[t]$

$ExactlyOnceCommit \triangleq$
$\forall\, r \in Requests :$
$\land\, commit_count[r] \leq 1$
$\land\, (jobs[r].state = \text{“ACCEPTED”}) \equiv (commit_count[r] = 1)$

$FailedImpliesExhausted \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state = \text{“FAILED”} \Rightarrow jobs[r].attempts = MaxAttempts$

$LeaseOwnership \triangleq$
$\forall\, r \in Requests :$
$\land\, jobs[r].state \in Active \Rightarrow jobs[r].owner \in Workers$
$\land\, jobs[r].state \notin Active \Rightarrow jobs[r].owner = NONE$

$QueueConsistency \triangleq$
$\forall\, r \in Requests :$
$\land\, jobs[r].state = \text{“QUEUED”} \Rightarrow QCount(r) = 1$
$\land\, jobs[r].state \neq \text{“QUEUED”} \Rightarrow QCount(r) = 0$

$FlightIsLocal \triangleq$
$\forall\, r \in Requests :$
$ProtocolOf[r] = \text{“FLIGHT”} \Rightarrow$
$\land\, jobs[r].state \in \{\text{“IDLE”}, \text{“ACCEPTED”}, \text{“UNAUTHORIZED”}\}$
$\land\, jobs[r].owner = NONE$
$\land\, QCount(r) = 0$

$SnapshotBound \triangleq$
$\forall\, r \in Requests :$
$\land\, jobs[r].state \notin \{\text{“IDLE”}, \text{“UNAUTHORIZED”}, \text{“CACHED”}\} \Rightarrow$
$jobs[r].version \in 1 \,..\, catalog_snapshot[jobs[r].tenant]$
$\land\, client_received[r].delivered \Rightarrow$
$client_received[r].version$
$\in 1 \,..\, catalog_snapshot[AuthenticatedTenantOf[r]]$
end define ;

fair process $ClientProcess \in Requests$
variables $req_version = 0$, $proxy = NONE$ **;**
begin
$Start$:
if $\neg MutantSkipTenantCheck \land AuthenticatedTenantOf[self] \neq RequestedTenantOf[self]$ **then**

$jobs[self] := Job_Unauthorized(jobs[self])$;
goto *Finish* ;
end if ;

BindSnapshot:
$req_version := catalog_snapshot[EffTenant(self)]$;
$proxy := hash_ring[EffTenant(self),\ QueryOf[self]]$;

CheckCache:
if $cache[proxy][EffTenant(self)][QueryOf[self]][req_version][SecurityContextOf[self]].valid$ **then**
$client_received[self] := Client_Deliver(client_received[self],$
$cache[proxy][EffTenant(self)][QueryOf[self]][req_version][SecurityContextOf[self]].blob_id,$
$EffTenant(self),\ QueryOf[self],\ req_version,\ SecurityContextOf[self])$;

$jobs[self] := Job_Cached(jobs[self])$; Marks job as terminating via Cache Short-Circuit
goto *Finish* ;
end if ;

InitializeJob:
$jobs[self] := Job_Init(jobs[self],\ EffTenant(self),\ SecurityContextOf[self],\ req_version)$;

Route:
if $ProtocolOf[self] =$ "FLIGHT" **then goto** *ExecuteFlight* ; **else goto** *EnqueueREST* ; **end if** ;

ExecuteFlight:
await $available_blobs \neq \{\}$;
with $b \in available_blobs$ **do**
$available_blobs := available_blobs \setminus \{b\}$;
$blob_storage[b] := Blob_Flight(jobs[self].tenant,\ jobs[self].query,\ jobs[self].version,\ jobs[self].security_context$
$jobs[self] := Job_Flight_Accepted(jobs[self],\ b)$;
$commit_count[self] := commit_count[self] + 1$;
$client_received[self] := Client_Deliver(client_received[self],\ b,\ jobs[self].tenant,\ jobs[self].query,\ jobs[self].ver$

if $FlightPopulatesCache \wedge jobs[self].version = catalog_snapshot[jobs[self].tenant]$ **then**
$cache[proxy][jobs[self].tenant][jobs[self].query]$
$[jobs[self].version][jobs[self].security_context] := ValidCache(b)$;
end if ;
end with ;
goto *Finish* ;

EnqueueREST:
$jobs[self] := Job_Enqueue(jobs[self])$;
$task_queue := Append(task_queue,\ self)$;

AwaitREST:

await $jobs[self].state \in Terminal$;
if $jobs[self].state =$ "ACCEPTED" **then**
$client_received[self] := Client_Deliver(client_received[self], jobs[self].blob_ptr, jobs[self].tenant, jobs[self].que$
end if ;

Finish:
if $client_received[self].delivered$ **then**
ReleaseBlob:
$client_received[self] := Client_Release(client_received[self])$;
end if ;
end process ;

fair process $ComputeWorker \in Workers$
variables $curr_req = NONE$, $curr_epoch = 0$, $curr_blob = NONE$, $curr_proxy = NONE$;
begin
ClaimTask:
while TRUE **do**
await $Len(task_queue) > 0 \land worker_status[self] =$ "ACTIVE" ;
with $r = Head(task_queue)$ **do**
$task_queue := Tail(task_queue)$;
$curr_req := r$;
$curr_blob := NONE$;
if $jobs[r].state =$ "QUEUED" **then**
$curr_epoch := jobs[r].lease_epoch + 1$;
$jobs[r] := Job_Claim(jobs[r], self)$;
else
$curr_epoch := 0$;
end if ;
end with ;

StartExecution:
if $curr_epoch > 0 \land worker_status[self] =$ "ACTIVE" $\land HoldsLease(curr_req, curr_epoch, self,$ "CLAIMED") **t**
$jobs[curr_req] := Job_Run(jobs[curr_req])$;
else goto *ClaimTask* ; **end if** ;

AllocateBlob:
await $worker_status[self] \neq$ "ACTIVE" $\lor \neg HoldsLease(curr_req, curr_epoch, self,$ "RUNNING") $\lor available_b$
if $worker_status[self] =$ "ACTIVE" $\land HoldsLease(curr_req, curr_epoch, self,$ "RUNNING") $\land available_blobs \neq$
with $b \in available_blobs$ **do**
$available_blobs := available_blobs \setminus \{b\}$;

$curr_blob := b$;
$blob_storage[b] := Blob_Stage(curr_req,\ curr_epoch,\ jobs[curr_req].tenant,\ jobs[curr_req].query,\ jobs[curr_req$
end with ;
else goto *ClaimTask* ; **end if** ;

PublishPointer:
if $worker_status[self] =$ "ACTIVE" $\wedge\ HoldsLease(curr_req,\ curr_epoch,\ self,$ "RUNNING") **then**
$jobs[curr_req] := Job_Stage(jobs[curr_req],\ curr_blob)$;
else goto *ClaimTask* ; **end if** ;

CommitJob:
if $worker_status[self] =$ "ACTIVE" $\wedge\ HoldsLease(curr_req,\ curr_epoch,\ self,$ "STAGED") **then**
$jobs[curr_req] := Job_Accept(jobs[curr_req])$;
$blob_storage[curr_blob] := Blob_Commit(blob_storage[curr_blob])$;
$commit_count[curr_req] := commit_count[curr_req] + 1$;
else goto *ClaimTask* ; **end if** ;

PopulateCache:
if $worker_status[self] =$ "ACTIVE"
$\wedge\ jobs[curr_req].state =$ "ACCEPTED"
$\wedge\ jobs[curr_req].blob_ptr = curr_blob$
$\wedge\ blob_storage[curr_blob].allocated$
$\wedge\ \neg blob_storage[curr_blob].staged$
$\wedge\ BlobMatches(curr_blob,\ jobs[curr_req].tenant,\ jobs[curr_req].query,$
$jobs[curr_req].version,\ jobs[curr_req].security_context)$
$\wedge\ jobs[curr_req].version = catalog_snapshot[jobs[curr_req].tenant]$ **then**

$curr_proxy := hash_ring[jobs[curr_req].tenant,\ jobs[curr_req].query]$;

$cache[curr_proxy][jobs[curr_req].tenant][jobs[curr_req].query]$
$[jobs[curr_req].version][jobs[curr_req].security_context] := ValidCache(curr_blob)$;
end if ;
end while ;
end process ;

fair process *KedaAutoscaler* = "KedaAutoscaler"
begin
ScaleLoop:
while TRUE **do**
either
await $Len(task_queue) \geq ScaleOutThreshold$
$\wedge\ \exists\, w \in Workers : worker_status[w] =$ "OFFLINE" ;

with $w \in \{x \in Workers : worker_status[x] =$ "OFFLINE"$\}$ **do**
$worker_status[w] :=$ "ACTIVE" **;**
end with ;
or
await $Len(task_queue) = 0$
$\wedge \exists\, w \in Workers : worker_status[w] =$ "ACTIVE"
$\wedge \forall\, r \in Requests : \neg(jobs[r].owner = w \wedge jobs[r].state \in Active)$ **;**
with $w \in \{x \in Workers : worker_status[x] =$ "ACTIVE"
$\wedge \forall\, r \in Requests : \neg(jobs[r].owner = x \wedge jobs[r].state \in Active)\}$ **do**
$worker_status[w] :=$ "OFFLINE" **;**
end with ;
end either ;
end while ;
end process ;

process $TimeoutInjector =$ "TimeoutInjector"
begin
$InjectLoop$:
while $timeouts < MaxTimeouts$ **do**
with $r \in \{x \in Requests : jobs[x].state \in Active \wedge worker_status[jobs[x].owner] =$ "ACTIVE" $\wedge \neg lease_expired$
$lease_expired[r] :=$ TRUE **;**
$timeouts := timeouts + 1$ **;**
end with ;
end while ;
end process ;

fair process $LeaseManager =$ "LeaseManager"
begin
$LeaseLoop$:
while TRUE **do**
either

with $r \in ExpirableJobs$ **do**
if $jobs[r].attempts < MaxAttempts$ **then**
$jobs[r] := Job_Retry(jobs[r])$ **;**
$task_queue := Append(task_queue,\, r)$ **;**
else
$jobs[r] := Job_Fail(jobs[r])$ **;**
end if ;

```
lease_expired[r] := FALSE ;
end with ;
or

with r ∈ {x ∈ Requests : jobs[x].state ∉ Active ∧ lease_expired[x]} do
lease_expired[r] := FALSE ;
end with ;
end either ;
end while ;
end process ;



process Crasher = "Crasher"
begin
CrashLoop:
while crashes < MaxCrashes do
await ∃ w ∈ Workers : worker_status[w] = "ACTIVE" ;
with w ∈ {x ∈ Workers : worker_status[x] = "ACTIVE"} do
worker_status[w] := "CRASHED" ;
crashes := crashes + 1 ;
end with ;
end while ;
end process ;

fair process Recoverer = "Recoverer"
begin
RecoverLoop:
while TRUE do
await ∃ w ∈ Workers : worker_status[w] = "CRASHED" ;
with w ∈ {x ∈ Workers : worker_status[x] = "CRASHED"} do
worker_status[w] := "OFFLINE" ;
end with ;
end while ;
end process ;



fair process GarbageCollector = "GarbageCollector"
begin
GCLoop:
while TRUE do
with b ∈ GCable do
```

```
blob_storage[b] := FreeBlob ;
available_blobs := available_blobs ∪ {b} ;
end with ;
end while ;
end process ;

fair process CacheEvictor = "CacheEvictor"
begin
EvictLoop:
while TRUE do
with e ∈ CachedKeys do
cache[e[1]][e[2]][e[3]][e[4]][e[5]] := NoEntry ;
end with ;
end while ;
end process ;

fair process LakehouseETL = "LakehouseETL"
begin
ETLCommit:
while TRUE do
with t ∈ {x ∈ Tenants : catalog_snapshot[x] < MaxSnapshot} do
cache := InvalidateCache(cache, t, catalog_snapshot[t] + 1) ;
catalog_snapshot[t] := catalog_snapshot[t] + 1 ;
end with ;
end while ;
end process ;

end algorithm ;

VARIABLES pc, hash_ring, catalog_snapshot, cache, task_queue, jobs,
blob_storage, available_blobs, worker_status, client_received,
commit_count, lease_expired, crashes, timeouts


Pending ≜ {"QUEUED", "CLAIMED", "RUNNING", "STAGED"}
Active ≜ {"CLAIMED", "RUNNING", "STAGED"}
Terminal ≜ {"ACCEPTED", "FAILED", "UNAUTHORIZED", "CACHED"}
AllStates ≜ {"IDLE"} ∪ Pending ∪ Terminal

EffTenant(r) ≜
IF MutantSkipTenantCheck
 THEN RequestedTenantOf[r]
 ELSE AuthenticatedTenantOf[r]

HoldsLease(r, e, w, st) ≜
```

$\land\ jobs[r].lease_epoch = e$
$\land\ jobs[r].owner = w$
$\land\ jobs[r].state = st$

$QCount(r) \triangleq Cardinality(\{i \in 1\,..\,Len(task_queue) : task_queue[i] = r\})$

$BlobMatches(b,\ t,\ q,\ v,\ s) \triangleq$
$\land\ blob_storage[b].tenant = t$
$\land\ blob_storage[b].query = q$
$\land\ blob_storage[b].version = v$
$\land\ blob_storage[b].security_context = s$

$BlobCached(b) \triangleq$
$\exists\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \land cache[p][t][q][v][s].blob_id = b$

$BlobJobHeld(b) \triangleq$
$\exists\, r \in Requests : jobs[r].blob_ptr = b \land \neg client_received[r].released$

$BlobClientHeld(b) \triangleq$
$\exists\, r \in Requests : client_received[r].delivered$
$\land \neg client_received[r].released$
$\land\ client_received[r].blob_id = b$

$StagedOrphan(b) \triangleq jobs[blob_storage[b].req].lease_epoch \neq blob_storage[b].epoch$

$GCable \triangleq \{b \in Blobs :$
$\land\ blob_storage[b].allocated$
$\land$ IF $blob_storage[b].staged$
THEN $StagedOrphan(b)$
ELSE $\neg BlobCached(b) \land \neg BlobJobHeld(b) \land \neg BlobClientHeld(b)\}$

$ExpirableJobs \triangleq \{r \in Requests :$
$\land\ jobs[r].state \in Active$
$\land\ (worker_status[jobs[r].owner] \neq$ "ACTIVE" $\lor\ lease_expired[r])\}$

$CachedKeys \triangleq \{k \in ProxyNodes \times Tenants \times Queries \times (1\,..\,MaxSnapshot) \times SecCtxs :$
$cache[k[1]][k[2]][k[3]][k[4]][k[5]].valid\}$

$Job_Unauthorized(j) \triangleq$
$[j$ EXCEPT $!.state =$ "UNAUTHORIZED"$]$

$Job_Cached(j) \triangleq$
$[j \text{ EXCEPT } !.state = \text{“CACHED”}]$

$Job_Init(j,\ t,\ sc,\ v) \triangleq$
$[j \text{ EXCEPT } !.tenant = t,\ !.security_context = sc,\ !.version = v]$

$Job_Flight_Accepted(j,\ b) \triangleq$
$[j \text{ EXCEPT } !.state = \text{“ACCEPTED”},\ !.lease_epoch = 1,\ !.attempts = 1,$
$!.blob_ptr = b,\ !.rls_applied = \text{TRUE},\ !.executed_predicate_tenant = j.tenant]$

$Job_Enqueue(j) \triangleq$
$[j \text{ EXCEPT } !.state = \text{“QUEUED”},\ !.attempts = j.attempts + 1]$

$Job_Claim(j,\ w) \triangleq$
$[j \text{ EXCEPT } !.owner = w,\ !.state = \text{“CLAIMED”},\ !.lease_epoch = j.lease_epoch + 1]$

$Job_Run(j) \triangleq$
$[j \text{ EXCEPT } !.state = \text{“RUNNING”},\ !.rls_applied = \text{TRUE},$
$!.executed_predicate_tenant = j.tenant]$

$Job_Stage(j,\ b) \triangleq$
$[j \text{ EXCEPT } !.blob_ptr = b,\ !.state = \text{“STAGED”}]$

$Job_Accept(j) \triangleq$
$[j \text{ EXCEPT } !.state = \text{“ACCEPTED”},\ !.owner = NONE]$

$Job_Retry(j) \triangleq$
$[j \text{ EXCEPT } !.owner = NONE,\ !.state = \text{“QUEUED”},\ !.attempts = j.attempts + 1,$
$!.lease_epoch = j.lease_epoch + 1,\ !.blob_ptr = NONE,$
$!.rls_applied = \text{FALSE},\ !.executed_predicate_tenant = NONE]$

$Job_Fail(j) \triangleq$
$[j \text{ EXCEPT } !.owner = NONE,\ !.state = \text{“FAILED”},\ !.lease_epoch = j.lease_epoch + 1,$
$!.blob_ptr = NONE,\ !.rls_applied = \text{FALSE},\ !.executed_predicate_tenant = NONE]$

$Client_Deliver(cr,\ b,\ t,\ q,\ v,\ sc) \triangleq$
$[cr \text{ EXCEPT } !.delivered = \text{TRUE},\ !.blob_id = b,\ !.tenant = t,$
$!.query = q,\ !.version = v,\ !.security_context = sc]$

$Client_Release(cr) \triangleq$
$[cr \text{ EXCEPT } !.released = \text{TRUE}]$

$Blob_Flight(t,\ q,\ v,\ sc) \triangleq$
$[allocated \mapsto \text{TRUE},\ staged \mapsto \text{FALSE},\ req \mapsto NONE,\ epoch \mapsto 0,$
$tenant \mapsto t,\ query \mapsto q,\ version \mapsto v,\ security_context \mapsto sc]$

$Blob_Stage(req,\ ep,\ t,\ q,\ v,\ sc) \triangleq$
$[allocated \mapsto \text{TRUE},\ staged \mapsto \text{TRUE},\ req \mapsto req,\ epoch \mapsto ep,$
$tenant \mapsto t,\ query \mapsto q,\ version \mapsto v,\ security_context \mapsto sc]$

$Blob_Commit(b_val) \triangleq$
$[b_val \text{ EXCEPT } !.staged = \text{FALSE}]$

$ValidCache(b) \triangleq$
$[valid \mapsto \text{TRUE},\ blob_id \mapsto b]$

$InvalidateCache(old_cache,\ t,\ new_v) \triangleq$
$[p \in ProxyNodes \mapsto$
$[old_cache[p] \text{ EXCEPT } ![t] =$
$[q \in Queries \mapsto$
$[v \in 1\ldots MaxSnapshot \mapsto$
$[s \in SecCtxs \mapsto$
$\text{IF } v < new_v$
$\text{THEN } NoEntry$
$\text{ELSE } old_cache[p][t][q][v][s]]]]]]$

$TypeOK \triangleq$
$\land\ catalog_snapshot \in [Tenants \to 1\ldots MaxSnapshot]$
$\land\ worker_status \in [Workers \to \{\text{“OFFLINE”},\ \text{“ACTIVE”},\ \text{“CRASHED”}\}]$
$\land\ hash_ring \in [Tenants \times Queries \to ProxyNodes]$
$\land\ \forall\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1\ldots MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \in \text{BOOLEAN}$
$\land\ \forall\, r \in Requests :$
$\land\ jobs[r].state \in AllStates$
$\land\ jobs[r].attempts \in 0\ldots MaxAttempts$
$\land\ jobs[r].lease_epoch \in Nat$
$\land\ jobs[r].blob_ptr \in Blobs \cup \{NONE\}$
$\land\ commit_count[r] \in 0\ldots 1$
$\land\ client_received[r].delivered \Rightarrow client_received[r].blob_id \neq NONE$

$TenantBinding \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state \notin \{\text{“IDLE”},\ \text{“UNAUTHORIZED”},\ \text{“CACHED”}\} \Rightarrow$
$\land\ jobs[r].tenant = AuthenticatedTenantOf[r]$
$\land\ jobs[r].security_context = SecurityContextOf[r]$

$RLSEnforcement \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state \in \{\text{“RUNNING”},\ \text{“STAGED”},\ \text{“ACCEPTED”}\} \Rightarrow$
$\land\, jobs[r].rls_applied$
$\land\, jobs[r].executed_predicate_tenant = AuthenticatedTenantOf[r]$

$UnauthorizedIsInert \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state = \text{“UNAUTHORIZED”} \Rightarrow$
$\land\, jobs[r].blob_ptr = NONE$
$\land\, jobs[r].owner = NONE$
$\land\, jobs[r].attempts = 0$
$\land\, commit_count[r] = 0$
$\land\, \neg client_received[r].delivered$
$\land\, QCount(r) = 0$

$MismatchRejected \triangleq$
$\forall\, r \in Requests :$
$RequestedTenantOf[r] \neq AuthenticatedTenantOf[r] \Rightarrow$
$\land\, jobs[r].state \in \{\text{“IDLE”},\ \text{“UNAUTHORIZED”}\}$
$\land\, \neg client_received[r].delivered$

$TenantIsolation \triangleq$
$\forall\, r \in Requests :$
$client_received[r].delivered \Rightarrow$
$\land\, client_received[r].tenant = AuthenticatedTenantOf[r]$
$\land\, client_received[r].security_context = SecurityContextOf[r]$
$\land\, client_received[r].query = QueryOf[r]$

$DeliveredResultIntegrity \triangleq$
$\forall\, r \in Requests :$
$(client_received[r].delivered \land \neg client_received[r].released) \Rightarrow$
$\land\, blob_storage[client_received[r].blob_id].allocated$
$\land\, \neg blob_storage[client_received[r].blob_id].staged$
$\land\, BlobMatches(client_received[r].blob_id,$
$AuthenticatedTenantOf[r],\ QueryOf[r],$
$client_received[r].version,\ SecurityContextOf[r])$

$ClaimCheckIntegrity \triangleq$
$\forall\, r \in Requests :$
$(jobs[r].state = \text{“ACCEPTED”} \land \neg client_received[r].released) \Rightarrow$
$\land\, blob_storage[jobs[r].blob_ptr].allocated$
$\land\, \neg blob_storage[jobs[r].blob_ptr].staged$
$\land\, BlobMatches(jobs[r].blob_ptr,\ jobs[r].tenant,\ jobs[r].query,$
$jobs[r].version,\ jobs[r].security_context)$

$BlobPtrIffResultState \triangleq$
$\forall\, r \in Requests :$
$(jobs[r].blob_ptr \neq NONE) \equiv$
$(jobs[r].state \in \{\text{“STAGED”},\ \text{“ACCEPTED”}\})$

$NoSharedBlobPointer \triangleq$
$\forall\, r1,\ r2 \in Requests :$
$(r1 \neq r2 \land jobs[r1].blob_ptr \neq NONE \land jobs[r2].blob_ptr \neq NONE$
$\land \neg client_received[r1].released \land \neg client_received[r2].released) \Rightarrow$
$jobs[r1].blob_ptr \neq jobs[r2].blob_ptr$

$BlobPoolConsistency \triangleq$
$available_blobs = \{b \in Blobs : \neg blob_storage[b].allocated\}$

$StagedBlobWellFormed \triangleq$
$\forall\, b \in Blobs :$
$blob_storage[b].staged \Rightarrow$
$\land\ blob_storage[b].allocated$
$\land\ blob_storage[b].req \in Requests$
$\land\ jobs[blob_storage[b].req].lease_epoch \geq blob_storage[b].epoch$

$CacheIntegrity \triangleq$
$\forall\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \Rightarrow$
$\land\ blob_storage[cache[p][t][q][v][s].blob_id].allocated$
$\land \neg blob_storage[cache[p][t][q][v][s].blob_id].staged$
$\land\ BlobMatches(cache[p][t][q][v][s].blob_id,\ t,\ q,\ v,\ s)$

$CacheFreshness \triangleq$
$\forall\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \Rightarrow v = catalog_snapshot[t]$

$ExactlyOnceCommit \triangleq$
$\forall\, r \in Requests :$
$\land\ commit_count[r] \leq 1$
$\land\ (jobs[r].state = \text{“ACCEPTED”}) \equiv (commit_count[r] = 1)$

$FailedImpliesExhausted \triangleq$
$\forall\, r \in Requests :$
$jobs[r].state = \text{“FAILED”} \Rightarrow jobs[r].attempts = MaxAttempts$

$LeaseOwnership \triangleq$

$\forall\, r \in \mathit{Requests} :$
$\land\, \mathit{jobs}[r].\mathit{state} \in \mathit{Active} \Rightarrow \mathit{jobs}[r].\mathit{owner} \in \mathit{Workers}$
$\land\, \mathit{jobs}[r].\mathit{state} \notin \mathit{Active} \Rightarrow \mathit{jobs}[r].\mathit{owner} = \mathit{NONE}$

$\mathit{QueueConsistency} \triangleq$
$\forall\, r \in \mathit{Requests} :$
$\land\, \mathit{jobs}[r].\mathit{state} = \text{“QUEUED”} \Rightarrow \mathit{QCount}(r) = 1$
$\land\, \mathit{jobs}[r].\mathit{state} \neq \text{“QUEUED”} \Rightarrow \mathit{QCount}(r) = 0$

$\mathit{FlightIsLocal} \triangleq$
$\forall\, r \in \mathit{Requests} :$
$\mathit{ProtocolOf}[r] = \text{“FLIGHT”} \Rightarrow$
$\land\, \mathit{jobs}[r].\mathit{state} \in \{\text{“IDLE”},\ \text{“ACCEPTED”},\ \text{“UNAUTHORIZED”}\}$
$\land\, \mathit{jobs}[r].\mathit{owner} = \mathit{NONE}$
$\land\, \mathit{QCount}(r) = 0$

$\mathit{SnapshotBound} \triangleq$
$\forall\, r \in \mathit{Requests} :$
$\land\, \mathit{jobs}[r].\mathit{state} \notin \{\text{“IDLE”},\ \text{“UNAUTHORIZED”},\ \text{“CACHED”}\} \Rightarrow$
$\mathit{jobs}[r].\mathit{version} \in 1 \,..\, \mathit{catalog_snapshot}[\mathit{jobs}[r].\mathit{tenant}]$
$\land\, \mathit{client_received}[r].\mathit{delivered} \Rightarrow$
$\mathit{client_received}[r].\mathit{version}$
$\in 1 \,..\, \mathit{catalog_snapshot}[\mathit{AuthenticatedTenantOf}[r]]$

VARIABLES $\mathit{req_version}$, proxy, $\mathit{curr_req}$, $\mathit{curr_epoch}$, $\mathit{curr_blob}$, $\mathit{curr_proxy}$

$\mathit{vars} \triangleq \langle \mathit{pc}, \mathit{hash_ring}, \mathit{catalog_snapshot}, \mathit{cache}, \mathit{task_queue}, \mathit{jobs},$
$\mathit{blob_storage}, \mathit{available_blobs}, \mathit{worker_status}, \mathit{client_received},$
$\mathit{commit_count}, \mathit{lease_expired}, \mathit{crashes}, \mathit{timeouts}, \mathit{req_version}, \mathit{proxy},$
$\mathit{curr_req}, \mathit{curr_epoch}, \mathit{curr_blob}, \mathit{curr_proxy}\rangle$

$\mathit{ProcSet} \triangleq (\mathit{Requests}) \cup (\mathit{Workers}) \cup \{\text{“KedaAutoscaler”}\} \cup \{\text{“TimeoutInjector”}\} \cup \{\text{“LeaseManager”}\} \cup \{\text{“Crash}$

$\mathit{Init} \triangleq$ Global variables
$\land\, \mathit{hash_ring} \in [\mathit{Tenants} \times \mathit{Queries} \to \mathit{ProxyNodes}]$
$\land\, \mathit{catalog_snapshot} = [t \in \mathit{Tenants} \mapsto 1]$
$\land\, \mathit{cache} = [p \in \mathit{ProxyNodes} \mapsto$
$[t \in \mathit{Tenants} \mapsto$
$[q \in \mathit{Queries} \mapsto$
$[v \in 1 \,..\, \mathit{MaxSnapshot} \mapsto$
$[s \in \mathit{SecCtxs} \mapsto \mathit{NoEntry}]]]]]$
$\land\, \mathit{task_queue} = \langle\rangle$
$\land\, \mathit{jobs} = [r \in \mathit{Requests} \mapsto$
$[\mathit{state} \mapsto \text{“IDLE”},$
$\mathit{owner} \mapsto \mathit{NONE},$

$lease_epoch \mapsto 0,$
$attempts \mapsto 0,$
$requested_tenant \mapsto RequestedTenantOf[r],$
$tenant \mapsto NONE,$
$query \mapsto QueryOf[r],$
$security_context \mapsto NONE,$
$version \mapsto 0,$
$blob_ptr \mapsto NONE,$
$protocol \mapsto ProtocolOf[r],$
$rls_applied \mapsto \text{FALSE},$
$executed_predicate_tenant \mapsto NONE]]$
$\land\ blob_storage = [b \in Blobs \mapsto FreeBlob]$
$\land\ available_blobs = Blobs$
$\land\ worker_status = [w \in Workers \mapsto \text{“OFFLINE”}]$
$\land\ client_received = [r \in Requests \mapsto$
$[delivered \mapsto \text{FALSE},$
$released \mapsto \text{FALSE},$
$blob_id \mapsto NONE,$
$tenant \mapsto NONE,$
$query \mapsto NONE,$
$version \mapsto 0,$
$security_context \mapsto NONE]]$
$\land\ commit_count = [r \in Requests \mapsto 0]$
$\land\ lease_expired = [r \in Requests \mapsto \text{FALSE}]$
$\land\ crashes = 0$
$\land\ timeouts = 0$

$\land\ req_version = [self \in Requests \mapsto 0]$
$\land\ proxy = [self \in Requests \mapsto NONE]$

$\land\ curr_req = [self \in Workers \mapsto NONE]$
$\land\ curr_epoch = [self \in Workers \mapsto 0]$
$\land\ curr_blob = [self \in Workers \mapsto NONE]$
$\land\ curr_proxy = [self \in Workers \mapsto NONE]$
$\land\ pc = [self \in ProcSet \mapsto \text{CASE } self \in Requests \to \text{“Start”}$
$\square self \in Workers \to \text{“ClaimTask”}$
$\square self = \text{“KedaAutoscaler”} \to \text{“ScaleLoop”}$
$\square self = \text{“TimeoutInjector”} \to \text{“InjectLoop”}$
$\square self = \text{“LeaseManager”} \to \text{“LeaseLoop”}$
$\square self = \text{“Crasher”} \to \text{“CrashLoop”}$
$\square self = \text{“Recoverer”} \to \text{“RecoverLoop”}$
$\square self = \text{“GarbageCollector”} \to \text{“GCLoop”}$
$\square self = \text{“CacheEvictor”} \to \text{“EvictLoop”}$
$\square self = \text{“LakehouseETL”} \to \text{“ETLCommit”}]$

$Start(self) \triangleq \ \land pc[self] = \text{“Start”}$
$\land$ IF $\neg MutantSkipTenantCheck \land AuthenticatedTenantOf[self] \neq RequestedTenantOf[self]$
THEN $\land jobs' = [jobs$ EXCEPT $![self] = Job_Unauthorized(jobs[self])]$
$\land pc' = [pc$ EXCEPT $![self] = \text{“Finish”}]$
ELSE $\land pc' = [pc$ EXCEPT $![self] = \text{“BindSnapshot”}]$
$\land jobs' = jobs$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache, task_queue,$
$blob_storage, available_blobs, worker_status,$
$client_received, commit_count, lease_expired,$
$crashes, timeouts, req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$BindSnapshot(self) \triangleq \ \land pc[self] = \text{“BindSnapshot”}$
$\land req_version' = [req_version$ EXCEPT $![self] = catalog_snapshot[EffTenant(self)]]$
$\land proxy' = [proxy$ EXCEPT $![self] = hash_ring[EffTenant(self), QueryOf[self]]]$
$\land pc' = [pc$ EXCEPT $![self] = \text{“CheckCache”}]$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, jobs, blob_storage,$
$available_blobs, worker_status,$
$client_received, commit_count,$
$lease_expired, crashes, timeouts,$
$curr_req, curr_epoch, curr_blob,$
$curr_proxy\rangle$

$CheckCache(self) \triangleq \ \land pc[self] = \text{“CheckCache”}$
$\land$ IF $cache[proxy[self]][EffTenant(self)][QueryOf[self]][req_version[self]][SecurityContextOf[self]].valid$
THEN $\land client_received' = [client_received$ EXCEPT $![self] = Client_Deliver(client_received[self],$
$cache[proxy[self]][EffTenant(self)][QueryOf[self]][req_version[self]][SecurityContextOf[self]].blob_id,$
$EffTenant(self), QueryOf[self], req_version[self], SecurityContextOf[self])]$
$\land jobs' = [jobs$ EXCEPT $![self] = Job_Cached(jobs[self])]$
$\land pc' = [pc$ EXCEPT $![self] = \text{“Finish”}]$
ELSE $\land pc' = [pc$ EXCEPT $![self] = \text{“InitializeJob”}]$
$\land$ UNCHANGED $\langle jobs, client_received\rangle$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, blob_storage, available_blobs,$
$worker_status, commit_count, lease_expired,$
$crashes, timeouts, req_version, proxy,$
$curr_req, curr_epoch, curr_blob,$
$curr_proxy\rangle$

$InitializeJob(self) \triangleq \ \land pc[self] = \text{“InitializeJob”}$
$\land jobs' = [jobs$ EXCEPT $![self] = Job_Init(jobs[self], EffTenant(self), SecurityContextOf[self], req_version[se$
$\land pc' = [pc$ EXCEPT $![self] = \text{“Route”}]$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, blob_storage,$

$available_blobs$, $worker_status$,
$client_received$, $commit_count$,
$lease_expired$, $crashes$, $timeouts$,
$req_version$, $proxy$, $curr_req$,
$curr_epoch$, $curr_blob$, $curr_proxy\rangle$

$Route(self) \triangleq \ \wedge pc[self] = \text{“Route”}$
$\wedge$ IF $ProtocolOf[self] = \text{“FLIGHT”}$
THEN $\wedge pc' = [pc$ EXCEPT $![self] = \text{“ExecuteFlight”}]$
ELSE $\wedge pc' = [pc$ EXCEPT $![self] = \text{“EnqueueREST”}]$
$\wedge$ UNCHANGED $\langle hash_ring$, $catalog_snapshot$, $cache$, $task_queue$,
$jobs$, $blob_storage$, $available_blobs$,
$worker_status$, $client_received$, $commit_count$,
$lease_expired$, $crashes$, $timeouts$, $req_version$,
$proxy$, $curr_req$, $curr_epoch$, $curr_blob$,
$curr_proxy\rangle$

$ExecuteFlight(self) \triangleq \ \wedge pc[self] = \text{“ExecuteFlight”}$
$\wedge\ available_blobs \neq \{\}$
$\wedge \exists\, b \in available_blobs :$
$\wedge\ available_blobs' = available_blobs \setminus \{b\}$
$\wedge\ blob_storage' = [blob_storage$ EXCEPT $![b] = Blob_Flight(jobs[self].tenant,\ jobs[self].query,\ jobs[self].versio$
$\wedge\ jobs' = [jobs$ EXCEPT $![self] = Job_Flight_Accepted(jobs[self],\ b)]$
$\wedge\ commit_count' = [commit_count$ EXCEPT $![self] = commit_count[self] + 1]$
$\wedge\ client_received' = [client_received$ EXCEPT $![self] = Client_Deliver(client_received[self],\ b,\ jobs'[self].tenan$
$\wedge$ IF $FlightPopulatesCache \wedge jobs'[self].version = catalog_snapshot[jobs'[self].tenant]$
THEN $\wedge\ cache' = [cache$ EXCEPT $![proxy[self]][jobs'[self].tenant][jobs'[self].query]$
$[jobs'[self].version][jobs'[self].security_context] = ValidCache(b)]$
ELSE $\wedge$ TRUE
$\wedge\ cache' = cache$
$\wedge\ pc' = [pc$ EXCEPT $![self] = \text{“Finish”}]$
$\wedge$ UNCHANGED $\langle hash_ring$, $catalog_snapshot$, $task_queue$,
$worker_status$, $lease_expired$, $crashes$,
$timeouts$, $req_version$, $proxy$, $curr_req$,
$curr_epoch$, $curr_blob$, $curr_proxy\rangle$

$EnqueueREST(self) \triangleq \ \wedge pc[self] = \text{“EnqueueREST”}$
$\wedge\ jobs' = [jobs$ EXCEPT $![self] = Job_Enqueue(jobs[self])]$
$\wedge\ task_queue' = Append(task_queue,\ self)$
$\wedge\ pc' = [pc$ EXCEPT $![self] = \text{“AwaitREST”}]$
$\wedge$ UNCHANGED $\langle hash_ring$, $catalog_snapshot$, $cache$,
$blob_storage$, $available_blobs$,
$worker_status$, $client_received$,
$commit_count$, $lease_expired$, $crashes$,
$timeouts$, $req_version$, $proxy$, $curr_req$,

$curr_epoch$, $curr_blob$, $curr_proxy\rangle$

$AwaitREST(self) \triangleq \ \wedge pc[self] = \text{“AwaitREST”}$
$\wedge\ jobs[self].state \in Terminal$
$\wedge$ IF $jobs[self].state = \text{“ACCEPTED”}$
THEN $\wedge\ client_received' = [client_received$ EXCEPT $![self] = Client_Deliver(client_received[self], jobs[self].bl$
ELSE $\wedge$ TRUE
$\wedge$ UNCHANGED $client_received$
$\wedge\ pc' = [pc$ EXCEPT $![self] = \text{“Finish”}]$
$\wedge$ UNCHANGED $\langle hash_ring$, $catalog_snapshot$, $cache$,
$task_queue$, $jobs$, $blob_storage$,
$available_blobs$, $worker_status$,
$commit_count$, $lease_expired$, $crashes$,
$timeouts$, $req_version$, $proxy$, $curr_req$,
$curr_epoch$, $curr_blob$, $curr_proxy\rangle$

$Finish(self) \triangleq \ \wedge pc[self] = \text{“Finish”}$
$\wedge$ IF $client_received[self].delivered$
THEN $\wedge\ pc' = [pc$ EXCEPT $![self] = \text{“ReleaseBlob”}]$
ELSE $\wedge\ pc' = [pc$ EXCEPT $![self] = \text{“Done”}]$
$\wedge$ UNCHANGED $\langle hash_ring$, $catalog_snapshot$, $cache$, $task_queue$,
$jobs$, $blob_storage$, $available_blobs$,
$worker_status$, $client_received$, $commit_count$,
$lease_expired$, $crashes$, $timeouts$, $req_version$,
$proxy$, $curr_req$, $curr_epoch$, $curr_blob$,
$curr_proxy\rangle$

$ReleaseBlob(self) \triangleq \ \wedge pc[self] = \text{“ReleaseBlob”}$
$\wedge\ client_received' = [client_received$ EXCEPT $![self] = Client_Release(client_received[self])]$
$\wedge\ pc' = [pc$ EXCEPT $![self] = \text{“Done”}]$
$\wedge$ UNCHANGED $\langle hash_ring$, $catalog_snapshot$, $cache$,
$task_queue$, $jobs$, $blob_storage$,
$available_blobs$, $worker_status$,
$commit_count$, $lease_expired$, $crashes$,
$timeouts$, $req_version$, $proxy$, $curr_req$,
$curr_epoch$, $curr_blob$, $curr_proxy\rangle$

$ClientProcess(self) \triangleq \ Start(self) \vee BindSnapshot(self)$
$\vee\ CheckCache(self) \vee InitializeJob(self)$
$\vee\ Route(self) \vee ExecuteFlight(self)$
$\vee\ EnqueueREST(self) \vee AwaitREST(self)$
$\vee\ Finish(self) \vee ReleaseBlob(self)$

$ClaimTask(self) \triangleq \ \wedge pc[self] = \text{“ClaimTask”}$
$\wedge\ Len(task_queue) > 0 \wedge worker_status[self] = \text{“ACTIVE”}$

$\land$ LET $r \triangleq Head(task_queue)$IN
$\land\ task_queue' = Tail(task_queue)$
$\land\ curr_req' = [curr_req$ EXCEPT $![self] = r]$
$\land\ curr_blob' = [curr_blob$ EXCEPT $![self] = NONE]$
$\land$ IF $jobs[r].state =$ "QUEUED"
THEN $\land\ curr_epoch' = [curr_epoch$ EXCEPT $![self] = jobs[r].lease_epoch + 1]$
$\land\ jobs' = [jobs$ EXCEPT $![r] = Job_Claim(jobs[r], self)]$
ELSE $\land\ curr_epoch' = [curr_epoch$ EXCEPT $![self] = 0]$
$\land\ jobs' = jobs$
$\land\ pc' = [pc$ EXCEPT $![self] =$ "StartExecution"$]$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$blob_storage, available_blobs,$
$worker_status, client_received,$
$commit_count, lease_expired, crashes,$
$timeouts, req_version, proxy, curr_proxy\rangle$

$StartExecution(self) \triangleq \land\ pc[self] =$ "StartExecution"
$\land$ IF $curr_epoch[self] > 0 \land worker_status[self] =$ "ACTIVE" $\land HoldsLease(curr_req[self], curr_epoch[self], se$
THEN $\land\ jobs' = [jobs$ EXCEPT $![curr_req[self]] = Job_Run(jobs[curr_req[self]])]$
$\land\ pc' = [pc$ EXCEPT $![self] =$ "AllocateBlob"$]$
ELSE $\land\ pc' = [pc$ EXCEPT $![self] =$ "ClaimTask"$]$
$\land\ jobs' = jobs$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, blob_storage,$
$available_blobs, worker_status,$
$client_received, commit_count,$
$lease_expired, crashes, timeouts,$
$req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$AllocateBlob(self) \triangleq \land\ pc[self] =$ "AllocateBlob"
$\land\ worker_status[self] \neq$ "ACTIVE" $\lor \neg HoldsLease(curr_req[self], curr_epoch[self], self,$ "RUNNING") $\lor$ *availa*
$\land$ IF $worker_status[self] =$ "ACTIVE" $\land HoldsLease(curr_req[self], curr_epoch[self], self,$ "RUNNING") $\land$ *avai*
THEN $\land \exists\, b \in available_blobs :$
$\land\ available_blobs' = available_blobs \setminus \{b\}$
$\land\ curr_blob' = [curr_blob$ EXCEPT $![self] = b]$
$\land\ blob_storage' = [blob_storage$ EXCEPT $![b] = Blob_Stage(curr_req[self], curr_epoch[self], jobs[curr_req[self]$
$\land\ pc' = [pc$ EXCEPT $![self] =$ "PublishPointer"$]$
ELSE $\land\ pc' = [pc$ EXCEPT $![self] =$ "ClaimTask"$]$
$\land$ UNCHANGED $\langle blob_storage, available_blobs,$
$curr_blob\rangle$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, jobs, worker_status,$
$client_received, commit_count,$
$lease_expired, crashes, timeouts,$

$req_version$, $proxy$, $curr_req$, $curr_epoch$,
$curr_proxy\rangle$

$PublishPointer(self) \triangleq \ \wedge pc[self] = \text{“PublishPointer”}$
$\wedge$ IF $worker_status[self] = \text{“ACTIVE”} \wedge HoldsLease(curr_req[self], curr_epoch[self], self, \text{“RUNNING”})$
THEN $\wedge jobs' = [jobs$ EXCEPT $![curr_req[self]] = Job_Stage(jobs[curr_req[self]], curr_blob[self])]$
$\wedge pc' = [pc$ EXCEPT $![self] = \text{“CommitJob”}]$
ELSE $\wedge pc' = [pc$ EXCEPT $![self] = \text{“ClaimTask”}]$
$\wedge jobs' = jobs$
$\wedge$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, blob_storage,$
$available_blobs, worker_status,$
$client_received, commit_count,$
$lease_expired, crashes, timeouts,$
$req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$CommitJob(self) \triangleq \ \wedge pc[self] = \text{“CommitJob”}$
$\wedge$ IF $worker_status[self] = \text{“ACTIVE”} \wedge HoldsLease(curr_req[self], curr_epoch[self], self, \text{“STAGED”})$
THEN $\wedge jobs' = [jobs$ EXCEPT $![curr_req[self]] = Job_Accept(jobs[curr_req[self]])]$
$\wedge blob_storage' = [blob_storage$ EXCEPT $![curr_blob[self]] = Blob_Commit(blob_storage[curr_blob[self]])]$
$\wedge commit_count' = [commit_count$ EXCEPT $![curr_req[self]] = commit_count[curr_req[self]] + 1]$
$\wedge pc' = [pc$ EXCEPT $![self] = \text{“PopulateCache”}]$
ELSE $\wedge pc' = [pc$ EXCEPT $![self] = \text{“ClaimTask”}]$
$\wedge$ UNCHANGED $\langle jobs, blob_storage, commit_count\rangle$
$\wedge$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, available_blobs, worker_status,$
$client_received, lease_expired, crashes,$
$timeouts, req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$PopulateCache(self) \triangleq \ \wedge pc[self] = \text{“PopulateCache”}$
$\wedge$ IF $worker_status[self] = \text{“ACTIVE”}$
$\wedge jobs[curr_req[self]].state = \text{“ACCEPTED”}$
$\wedge jobs[curr_req[self]].blob_ptr = curr_blob[self]$
$\wedge blob_storage[curr_blob[self]].allocated$
$\wedge \neg blob_storage[curr_blob[self]].staged$
$\wedge BlobMatches(curr_blob[self], jobs[curr_req[self]].tenant, jobs[curr_req[self]].query,$
$jobs[curr_req[self]].version, jobs[curr_req[self]].security_context)$
$\wedge jobs[curr_req[self]].version = catalog_snapshot[jobs[curr_req[self]].tenant]$
THEN $\wedge curr_proxy' = [curr_proxy$ EXCEPT $![self] = hash_ring[jobs[curr_req[self]].tenant, jobs[curr_req[self]$
$\wedge cache' = [cache$ EXCEPT $![curr_proxy'[self]][jobs[curr_req[self]].tenant][jobs[curr_req[self]].query]$
$[jobs[curr_req[self]].version][jobs[curr_req[self]].security_context] = ValidCache(curr_blob[self])]$
ELSE $\wedge$ TRUE
$\wedge$ UNCHANGED $\langle cache, curr_proxy\rangle$

$\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“ClaimTask”}]$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ task_queue,$
$jobs,\ blob_storage,\ available_blobs,$
$worker_status,\ client_received,$
$commit_count,\ lease_expired,\ crashes,$
$timeouts,\ req_version,\ proxy,\ curr_req,$
$curr_epoch,\ curr_blob\rangle$

$ComputeWorker(self) \triangleq ClaimTask(self) \lor StartExecution(self)$
$\lor\ AllocateBlob(self) \lor PublishPointer(self)$
$\lor\ CommitJob(self) \lor PopulateCache(self)$

$ScaleLoop \triangleq\ \land\ pc[\text{“KedaAutoscaler”}] = \text{“ScaleLoop”}$
$\land\ \lor\ \land\ Len(task_queue) \geq ScaleOutThreshold$
$\land \exists\, w \in Workers : worker_status[w] = \text{“OFFLINE”}$
$\land \exists\, w \in \{x \in Workers : worker_status[x] = \text{“OFFLINE”}\} :$
$worker_status' = [worker_status \text{ EXCEPT } ![w] = \text{“ACTIVE”}]$
$\lor\ \land\ Len(task_queue) = 0$
$\land \exists\, w \in Workers : worker_status[w] = \text{“ACTIVE”}$
$\land \forall\, r \in Requests : \neg(jobs[r].owner = w \land jobs[r].state \in Active)$
$\land \exists\, w \in \{x \in Workers : worker_status[x] = \text{“ACTIVE”}$
$\land \forall\, r \in Requests : \neg(jobs[r].owner = x \land jobs[r].state \in Active)\} :$
$worker_status' = [worker_status \text{ EXCEPT } ![w] = \text{“OFFLINE”}]$
$\land\ pc' = [pc \text{ EXCEPT } ![\text{“KedaAutoscaler”}] = \text{“ScaleLoop”}]$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,\ task_queue,$
$jobs,\ blob_storage,\ available_blobs,$
$client_received,\ commit_count,\ lease_expired,$
$crashes,\ timeouts,\ req_version,\ proxy,\ curr_req,$
$curr_epoch,\ curr_blob,\ curr_proxy\rangle$

$KedaAutoscaler \triangleq ScaleLoop$

$InjectLoop \triangleq\ \land\ pc[\text{“TimeoutInjector”}] = \text{“InjectLoop”}$
$\land$ IF $timeouts < MaxTimeouts$
THEN $\land \exists\, r \in \{x \in Requests : jobs[x].state \in Active \land worker_status[jobs[x].owner] = \text{“ACTIVE”} \land \neg lease_ex$
$\land\ lease_expired' = [lease_expired \text{ EXCEPT } ![r] = \text{TRUE}]$
$\land\ timeouts' = timeouts + 1$
$\land\ pc' = [pc \text{ EXCEPT } ![\text{“TimeoutInjector”}] = \text{“InjectLoop”}]$
ELSE $\land\ pc' = [pc \text{ EXCEPT } ![\text{“TimeoutInjector”}] = \text{“Done”}]$
$\land$ UNCHANGED $\langle lease_expired,\ timeouts\rangle$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,\ task_queue,$
$jobs,\ blob_storage,\ available_blobs,$
$worker_status,\ client_received,\ commit_count,$
$crashes,\ req_version,\ proxy,\ curr_req,$
$curr_epoch,\ curr_blob,\ curr_proxy\rangle$

$TimeoutInjector \triangleq InjectLoop$

$LeaseLoop \triangleq \land pc[\text{“LeaseManager”}] = \text{“LeaseLoop”}$
$\land \lor \land \exists\, r \in ExpirableJobs :$
$\land$ IF $jobs[r].attempts < MaxAttempts$
THEN $\land jobs' = [jobs$ EXCEPT $![r] = Job_Retry(jobs[r])]$
$\land task_queue' = Append(task_queue, r)$
ELSE $\land jobs' = [jobs$ EXCEPT $![r] = Job_Fail(jobs[r])]$
$\land$ UNCHANGED $task_queue$
$\land lease_expired' = [lease_expired$ EXCEPT $![r] = \text{FALSE}]$
$\lor \land \exists\, r \in \{x \in Requests : jobs[x].state \notin Active \land lease_expired[x]\} :$
$lease_expired' = [lease_expired$ EXCEPT $![r] = \text{FALSE}]$
$\land$ UNCHANGED $\langle task_queue, jobs\rangle$
$\land pc' = [pc$ EXCEPT $![\text{“LeaseManager”}] = \text{“LeaseLoop”}]$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache, blob_storage,$
$available_blobs, worker_status, client_received,$
$commit_count, crashes, timeouts, req_version,$
$proxy, curr_req, curr_epoch, curr_blob,$
$curr_proxy\rangle$

$LeaseManager \triangleq LeaseLoop$

$CrashLoop \triangleq \land pc[\text{“Crasher”}] = \text{“CrashLoop”}$
$\land$ IF $crashes < MaxCrashes$
THEN $\land \exists\, w \in Workers : worker_status[w] = \text{“ACTIVE”}$
$\land \exists\, w \in \{x \in Workers : worker_status[x] = \text{“ACTIVE”}\} :$
$\land worker_status' = [worker_status$ EXCEPT $![w] = \text{“CRASHED”}]$
$\land crashes' = crashes + 1$
$\land pc' = [pc$ EXCEPT $![\text{“Crasher”}] = \text{“CrashLoop”}]$
ELSE $\land pc' = [pc$ EXCEPT $![\text{“Crasher”}] = \text{“Done”}]$
$\land$ UNCHANGED $\langle worker_status, crashes\rangle$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache, task_queue,$
$jobs, blob_storage, available_blobs,$
$client_received, commit_count, lease_expired,$
$timeouts, req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$Crasher \triangleq CrashLoop$

$RecoverLoop \triangleq \land pc[\text{“Recoverer”}] = \text{“RecoverLoop”}$
$\land \exists\, w \in Workers : worker_status[w] = \text{“CRASHED”}$
$\land \exists\, w \in \{x \in Workers : worker_status[x] = \text{“CRASHED”}\} :$
$worker_status' = [worker_status$ EXCEPT $![w] = \text{“OFFLINE”}]$
$\land pc' = [pc$ EXCEPT $![\text{“Recoverer”}] = \text{“RecoverLoop”}]$

$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,\ task_queue,$
$jobs,\ blob_storage,\ available_blobs,$
$client_received,\ commit_count,\ lease_expired,$
$crashes,\ timeouts,\ req_version,\ proxy,\ curr_req,$
$curr_epoch,\ curr_blob,\ curr_proxy\rangle$

$Recoverer \triangleq RecoverLoop$

$GCLoop \triangleq \land pc[\text{“GarbageCollector”}] = \text{“GCLoop”}$
$\land \exists\, b \in GCable :$
$\land\ blob_storage' = [blob_storage \text{ EXCEPT } ![b] = FreeBlob]$
$\land\ available_blobs' = (available_blobs \cup \{b\})$
$\land\ pc' = [pc \text{ EXCEPT } ![\text{“GarbageCollector”}] = \text{“GCLoop”}]$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,\ task_queue,\ jobs,$
$worker_status,\ client_received,\ commit_count,$
$lease_expired,\ crashes,\ timeouts,\ req_version,\ proxy,$
$curr_req,\ curr_epoch,\ curr_blob,\ curr_proxy\rangle$

$GarbageCollector \triangleq GCLoop$

$EvictLoop \triangleq \land pc[\text{“CacheEvictor”}] = \text{“EvictLoop”}$
$\land \exists\, e \in CachedKeys :$
$cache' = [cache \text{ EXCEPT } ![e[1]][e[2]][e[3]][e[4]][e[5]] = NoEntry]$
$\land\ pc' = [pc \text{ EXCEPT } ![\text{“CacheEvictor”}] = \text{“EvictLoop”}]$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ task_queue,\ jobs,$
$blob_storage,\ available_blobs,\ worker_status,$
$client_received,\ commit_count,\ lease_expired,$
$crashes,\ timeouts,\ req_version,\ proxy,\ curr_req,$
$curr_epoch,\ curr_blob,\ curr_proxy\rangle$

$CacheEvictor \triangleq EvictLoop$

$ETLCommit \triangleq \land pc[\text{“LakehouseETL”}] = \text{“ETLCommit”}$
$\land \exists\, t \in \{x \in Tenants : catalog_snapshot[x] < MaxSnapshot\} :$
$\land\ cache' = InvalidateCache(cache,\ t,\ catalog_snapshot[t] + 1)$
$\land\ catalog_snapshot' = [catalog_snapshot \text{ EXCEPT } ![t] = catalog_snapshot[t] + 1]$
$\land\ pc' = [pc \text{ EXCEPT } ![\text{“LakehouseETL”}] = \text{“ETLCommit”}]$
$\land$ UNCHANGED $\langle hash_ring,\ task_queue,\ jobs,\ blob_storage,$
$available_blobs,\ worker_status,\ client_received,$
$commit_count,\ lease_expired,\ crashes,\ timeouts,$
$req_version,\ proxy,\ curr_req,\ curr_epoch,$
$curr_blob,\ curr_proxy\rangle$

$LakehouseETL \triangleq ETLCommit$

$$
\begin{aligned}
&\mathit{Next} \triangleq \mathit{KedaAutoscaler} \lor \mathit{TimeoutInjector} \lor \mathit{LeaseManager} \lor \mathit{Crasher} \\
&\lor \mathit{Recoverer} \lor \mathit{GarbageCollector} \lor \mathit{CacheEvictor} \lor \mathit{LakehouseETL} \\
&\lor (\exists\, \mathit{self} \in \mathit{Requests} : \mathit{ClientProcess}(\mathit{self})) \\
&\lor (\exists\, \mathit{self} \in \mathit{Workers} : \mathit{ComputeWorker}(\mathit{self}))
\end{aligned}
$$

$$
\begin{aligned}
&\mathit{Spec} \triangleq \land \mathit{Init} \land \Box[\mathit{Next}]_{\mathit{vars}} \\
&\land \forall\, \mathit{self} \in \mathit{Requests} : \mathrm{WF}_{\mathit{vars}}(\mathit{ClientProcess}(\mathit{self})) \\
&\land \forall\, \mathit{self} \in \mathit{Workers} : \mathrm{WF}_{\mathit{vars}}(\mathit{ComputeWorker}(\mathit{self})) \\
&\land \mathrm{WF}_{\mathit{vars}}(\mathit{KedaAutoscaler}) \\
&\land \mathrm{WF}_{\mathit{vars}}(\mathit{LeaseManager}) \\
&\land \mathrm{WF}_{\mathit{vars}}(\mathit{Recoverer}) \\
&\land \mathrm{WF}_{\mathit{vars}}(\mathit{GarbageCollector}) \\
&\land \mathrm{WF}_{\mathit{vars}}(\mathit{CacheEvictor}) \\
&\land \mathrm{WF}_{\mathit{vars}}(\mathit{LakehouseETL})
\end{aligned}
$$